Parse, don't validate

2019-11-05 ⊙ functional programming, haskell, types

Historically, I've struggled to find a concise, simple way to explain what it means to practice type-driven design. Too often, when someone asks me "How did you come up with this approach?" I find I can't give them a satisfying answer. I know it didn't just come to me in a vision—I have an iterative design process that doesn't require plucking the "right" approach out of thin air—yet I haven't been very successful in communicating that process to others.

However, about a month ago, I was reflecting on Twitter about the differences I experienced parsing JSON in statically- and dynamically-typed languages, and finally, I realized what I was looking for. Now I have a single, snappy slogan that encapsulates what type-driven design means to me, and better yet, it's only three words long:

Parse, don't validate.

The essence of type-driven design

Alright, I'll confess: unless you already know what type-driven design is, my catchy slogan probably doesn't mean all that much to you. Fortunately, that's what the remainder of this blog post is for. I'm going to explain precisely what I mean in gory detail—but first, we need to practice a little wishful thinking.

The realm of possibility

One of the wonderful things about static type systems is that they can make it possible, and sometimes even easy, to answer questions like "is it possible to write this function?" For an extreme example, consider the following Haskell type signature:

```
foo :: Integer -> Void
```

Is it possible to implement `foo`? Trivially, the answer is *no*, as `Void` is a type that contains no values, so it's impossible for *any* function to produce a value of type `Void`.[1] That example is pretty boring, but the question gets much more interesting if we choose a more realistic example:

```
head :: [a] -> a
```

This function returns the first element from a list. Is it possible to implement? It certainly doesn't sound like it does anything very complicated, but if we attempt to implement it, the compiler won't be satisfied:

```
head :: [a] -> a
head (x:_) = x
```

```
warning: [-Wincomplete-patterns]
    Pattern match(es) are non-exhaustive
    In an equation for 'head': Patterns not matched: []
```

This message is helpfully pointing out that our function is *partial*, which is to say it is not defined for all possible inputs. Specifically, it is not defined when the input is `[]`, the empty list. This makes sense, as it isn't possible to return the first element of a list if the list is empty—there's no element to return! So, remarkably, we learn this function isn't possible to implement, either.

Turning partial functions total

To someone coming from a dynamically-typed background, this might seem perplexing. If we have a list, we might very well want to get the first element in it. And indeed, the operation of "getting the first element of a list" isn't impossible in Haskell, it just requires a little extra ceremony. There are two different ways to fix the `head` function, and we'll start with the simplest one.

Managing expectations

As established, `head` is partial because there is no element to return if the list is empty: we've made a promise we cannot possibly fulfill. Fortunately, there's an easy solution to that dilemma: we can weaken our promise. Since we cannot guarantee the caller an element of the list, we'll have to practice a little expectation management: we'll do our best return an element if we can, but we reserve the right to return nothing at all. In Haskell, we express this possibility using the `Maybe` type:

```
head :: [a] -> Maybe a
```

This buys us the freedom we need to implement `head`—it allows us to return `Nothing` when we discover we can't produce a value of type `a` after all:

```
head :: [a] -> Maybe a
head (x:_) = Just x
head []    = Nothing
```

Problem solved, right? For the moment, yes… but this solution has a hidden cost.

Returning `Maybe` is undoubtably convenient when we're *implementing* `head`. However, it becomes significantly less convenient when we want to actually use it! Since `head` always has the potential to return `Nothing`, the burden falls upon its callers to handle that possibility, and sometimes that passing of the buck can be incredibly frustrating. To see why, consider the following code:

```
getConfigurationDirectories :: IO [FilePath]
getConfigurationDirectories = do
  configDirsString <- getEnv "CONFIG_DIRS"
  let configDirsList = split ',' configDirsString
  when (null configDirsList) $
    throwIO $ userError "CONFIG_DIRS cannot be empty"
  pure configDirsList

main :: IO ()
main = do
  configDirs <- getConfigurationDirectories
  case head configDirs of
    Just cacheDir -> initializeCache cacheDir
    Nothing -> error "should never happen; already checked configDirs is non-empty"
```

When `getConfigurationDirectories` retrieves a list of file paths from the environment, it proactively checks that the list is non-empty. However, when we use `head` in `main` to get the first element of the list, the `Maybe FilePath` result still requires us to handle a `Nothing` case that we know will never happen! This is terribly bad for several reasons:

1. First, it's just annoying. We already checked that the list is non-empty, why do we have to clutter our code with another redundant check?

2. Second, it has a potential performance cost. Although the cost of the redundant check is trivial in this particular example, one could imagine a more complex scenario where the redundant checks could add up, such as if they were happening in a tight loop.

3. Finally, and worst of all, this code is a bug waiting to happen! What if `getConfigurationDirectories` were modified to stop checking that the list is empty, intentionally or unintentionally? The programmer might not remember to update `main`, and suddenly the "impossible" error becomes not only possible, but probable.

The need for this redundant check has essentially forced us to punch a hole in our type system. If we could statically *prove* the `Nothing` case impossible, then a modification to `getConfigurationDirectories` that stopped checking if the list was empty would invalidate the proof and trigger a compile-time failure. However, as-written, we're forced to rely on a test suite or manual inspection to catch the bug.

Paying it forward

Clearly, our modified version of `head` leaves some things to be desired. Somehow, we'd like it to be smarter: if we already checked that the list was non-empty, `head` should unconditionally return the first element without forcing us to handle the case we know is impossible. How can we do that?

Let's look at the original (partial) type signature for `head` again:

```
head :: [a] -> a
```

The previous section illustrated that we can turn that partial type signature into a total one by weakening the promise made in the return type. However, since we don't want to do that, there's only one thing left that can be changed: the argument type (in this case, `[a]`). Instead of weakening the return type, we can *strengthen* the argument type, eliminating the possibility of `head` ever being called on an empty list in the first place.

To do this, we need a type that represents non-empty lists. Fortunately, the existing `NonEmpty` type from `Data.List.NonEmpty` is exactly that. It has the following definition:

```
data NonEmpty a = a :| [a]
```

Note that `NonEmpty a` is really just a tuple of an `a` and an ordinary, possibly-empty `[a]`. This conveniently models a non-empty list by storing the first element of the list separately from the list's tail: even if the `[a]` component is `[]`, the `a` component must always be present. This makes `head` completely trivial to implement:[2]

```
head :: NonEmpty a -> a
head (x:|_) = x
```

Unlike before, GHC accepts this definition without complaint—this definition is *total*, not partial. We can update our program to use the new implementation:

```
getConfigurationDirectories :: IO (NonEmpty FilePath)
getConfigurationDirectories = do
  configDirsString <- getEnv "CONFIG_DIRS"
  let configDirsList = split ' ' configDirsString
  case nonEmpty configDirsList of
    Just nonEmptyConfigDirsList -> pure nonEmptyConfigDirsList
    Nothing -> throwIO $ userError "CONFIG_DIRS cannot be empty"

main :: IO ()
main = do
```

```
  configDirs <- getConfigurationDirectories
  initializeCache (head configDirs)
```

Note that the redundant check in `main` is now completely gone! Instead, we perform the check exactly once, in `getConfigurationDirectories`. It constructs a `NonEmpty a` from a `[a]` using the `nonEmpty` function from `Data.List.NonEmpty`, which has the following type:

```
nonEmpty :: [a] -> Maybe (NonEmpty a)
```

The `Maybe` is still there, but this time, we handle the `Nothing` case very early in our program: right in the same place we were already doing the input validation. Once that check has passed, we now have a `NonEmpty FilePath` value, which preserves (in the type system!) the knowledge that the list really is non-empty. Put another way, you can think of a value of type `NonEmpty a` as being like a value of type `[a]`, plus a *proof* that the list is non-empty.

By strengthening the type of the argument to `head` instead of weakening the type of its result, we've completely eliminated all the problems from the previous section:

- The code has no redundant checks, so there can't be any performance overhead.
- Furthermore, if `getConfigurationDirectories` changes to stop checking that the list is non-empty, its return type must change, too. Consequently, `main` will fail to typecheck, alerting us to the problem before we even run the program!

What's more, it's trivial to recover the old behavior of `head` from the new one by composing `head` with `nonEmpty`:

```
head' :: [a] -> Maybe a
head' = fmap head . nonEmpty
```

Note that the inverse is *not* true: there is no way to obtain the new version of `head` from the old one. All in all, the second approach is superior on all axes.

The power of parsing

You may be wondering what the above example has to do with the title of this blog post. After all, we only examined two different ways to validate that a list was non-empty—no parsing in sight. That interpretation isn't wrong, but I'd like to propose another perspective: in my mind, the difference between validation and parsing lies almost entirely in how information is preserved. Consider the following pair of functions:

```
validateNonEmpty :: [a] -> IO ()
validateNonEmpty (_:_) = pure ()
validateNonEmpty [] = throwIO $ userError "list cannot be empty"

parseNonEmpty :: [a] -> IO (NonEmpty a)
parseNonEmpty (x:xs) = pure (x:|xs)
parseNonEmpty [] = throwIO $ userError "list cannot be empty"
```

These two functions are nearly identical: they check if the provided list is empty, and if it is, they abort the program with an error message. The difference lies entirely in the return type: `validateNonEmpty` always returns `()`, the type that contains no information, but `parseNonEmpty` returns `NonEmpty a`, a refinement of the input type that preserves the knowledge gained in the type system. Both of these functions check the same thing, but `parseNonEmpty` gives the caller access to the information it learned, while `validateNonEmpty` just throws it away.

These two functions elegantly illustrate two different perspectives on the role of a static type system: `validateNonEmpty` obeys the typechecker well enough, but only `parseNonEmpty` takes full advantage of it. If you see why `parseNonEmpty` is preferable, you understand what I mean by the mantra "parse, don't validate." Still, perhaps you are skeptical of `parseNonEmpty`'s name. Is it really *parsing* anything, or is it merely validating its input and returning a result? While the precise definition of what it means to parse or validate something is debatable, I believe `parseNonEmpty` is a bona-fide parser (albeit a particularly simple one).

Consider: what is a parser? Really, a parser is just a function that consumes less-structured input and produces more-structured output. By its very nature, a parser is a partial function—some values in the domain do not correspond to any value in the range—so all parsers must have some notion of failure. Often, the input to a parser is text, but this is by no means a requirement, and `parseNonEmpty` is a perfectly cromulent parser: it parses lists into non-empty lists, signaling failure by terminating the program with an error message.

Under this flexible definition, parsers are an incredibly powerful tool: they allow discharging checks on input up-front, right on the boundary between a program and the outside world, and once those checks have been performed, they never need to be checked again! Haskellers are well-aware of this power, and they use many different types of parsers on a regular basis:

- The aeson library provides a `Parser` type that can be used to parse JSON data into domain types.
- Likewise, optparse-applicative provides a set of parser combinators for parsing command-line arguments.
- Database libraries like persistent and postgresql-simple have a mechanism for parsing values held in an external data store.

- The servant ecosystem is built around parsing Haskell datatypes from path components, query parameters, HTTP headers, and more.

The common theme between all these libraries is that they sit on the boundary between your Haskell application and the external world. That world doesn't speak in product and sum types, but in streams of bytes, so there's no getting around a need to do some parsing. Doing that parsing up front, before acting on the data, can go a long way toward avoiding many classes of bugs, some of which might even be security vulnerabilities.

One drawback to this approach of parsing everything up front is that it sometimes requires values be parsed long before they are actually used. In a dynamically-typed language, this can make keeping the parsing and processing logic in sync a little tricky without extensive test coverage, much of which can be laborious to maintain. However, with a static type system, the problem becomes marvelously simple, as demonstrated by the `NonEmpty` example above: if the parsing and processing logic go out of sync, the program will fail to even compile.

The danger of validation

Hopefully, by this point, you are at least somewhat sold on the idea that parsing is preferable to validation, but you may have lingering doubts. Is validation really so bad if the type system is going to force you to do the necessary checks eventually anyway? Maybe the error reporting will be a little bit worse, but a bit of redundant checking can't hurt, right?

Unfortunately, it isn't so simple. Ad-hoc validation leads to a phenomenon that the language-theoretic security field calls *shotgun parsing*. In the 2016 paper, The Seven Turrets of Babel: A Taxonomy of LangSec Errors and How to Expunge Them, its authors provide the following definition:

> Shotgun parsing is a programming antipattern whereby parsing and input-validating code is mixed with and spread across processing code—throwing a cloud of checks at the input, and hoping, without any systematic justification, that one or another would catch all the "bad" cases.

They go on to explain the problems inherent to such validation techniques:

> Shotgun parsing necessarily deprives the program of the ability to reject invalid input instead of processing it. Late-discovered errors in an input stream will result in some portion of invalid input having been processed, with the consequence that program state is difficult to accurately predict.

In other words, a program that does not parse all of its input up front runs the risk of acting upon a valid portion of the input, discovering a different portion is invalid, and suddenly needing to roll back

whatever modifications it already executed in order to maintain consistency. Sometimes this is possible—such as rolling back a transaction in an RDBMS—but in general it may not be.

It may not be immediately apparent what shotgun parsing has to do with validation—after all, if you do all your validation up front, you mitigate the risk of shotgun parsing. The problem is that validation-based approaches make it extremely difficult or impossible to determine if everything was actually validated up front or if some of those so-called "impossible" cases might actually happen. The entire program must assume that raising an exception anywhere is not only possible, it's regularly necessary.

Parsing avoids this problem by stratifying the program into two phases—parsing and execution—where failure due to invalid input can only happen in the first phase. The set of remaining failure modes during execution is minimal by comparison, and they can be handled with the tender care they require.

Parsing, not validating, in practice

So far, this blog post has been something of a sales pitch. "You, dear reader, ought to be parsing!" it says, and if I've done my job properly, at least some of you are sold. However, even if you understand the "what" and the "why," you might not feel especially confident about the "how."

My advice: focus on the datatypes.

Suppose you are writing a function that accepts a list of tuples representing key-value pairs, and you suddenly realize you aren't sure what to do if the list has duplicate keys. One solution would be to write a function that asserts there aren't any duplicates in the list:

```
checkNoDuplicateKeys :: (MonadError AppError m, Eq k) => [(k, v)] -> m ()
```

However, this check is fragile: it's extremely easy to forget. Because its return value is unused, it can always be omitted, and the code that needs it would still typecheck. A better solution is to choose a data structure that disallows duplicate keys by construction, such as a `Map`. Adjust your function's type signature to accept a `Map` instead of a list of tuples, and implement it as you normally would.

Once you've done that, the call site of your new function will likely fail to typecheck, since it is still being passed a list of tuples. If the caller was given the value via one of its arguments, or if it received it from the result of some other function, you can continue updating the type from list to `Map`, all the way up the call chain. Eventually, you will either reach the location the value is created, or you'll find a place where duplicates actually ought to be allowed. At that point, you can insert a call to a modified version of `checkNoDuplicateKeys`:

```
checkNoDuplicateKeys :: (MonadError AppError m, Eq k) => [(k, v)] -> m (Map k v)
```

Now the check *cannot* be omitted, since its result is actually necessary for the program to proceed!

This hypothetical scenario highlights two simple ideas:

1. **Use a data structure that makes illegal states unrepresentable.** Model your data using the most precise data structure you reasonably can. If ruling out a particular possibility is too hard using the encoding you are currently using, consider alternate encodings that can express the property you care about more easily. Don't be afraid to refactor.

2. **Push the burden of proof upward as far as possible, but no further.** Get your data into the most precise representation you need as quickly as you can. Ideally, this should happen at the boundary of your system, before *any* of the data is acted upon.[3]

   If one particular code branch eventually requires a more precise representation of a piece of data, parse the data into the more precise representation as soon as the branch is selected. Use sum types judiciously to allow your datatypes to reflect and adapt to control flow.

In other words, write functions on the data representation you *wish* you had, not the data representation you are given. The design process then becomes an exercise in bridging the gap, often by working from both ends until they meet somewhere in the middle. Don't be afraid to iteratively adjust parts of the design as you go, since you may learn something new during the refactoring process!

Here are a handful of additional points of advice, arranged in no particular order:

- **Let your datatypes inform your code, don't let your code control your datatypes.** Avoid the temptation to just stick a `Bool` in a record somewhere because it's needed by the function you're currently writing. Don't be afraid to refactor code to use the right data representation—the type system will ensure you've covered all the places that need changing, and it will likely save you a headache later.

- **Treat functions that return `m ()` with deep suspicion.** Sometimes these are genuinely necessary, as they may perform an imperative effect with no meaningful result, but if the primary purpose of that effect is raising an error, it's likely there's a better way.

- **Don't be afraid to parse data in multiple passes.** Avoiding shotgun parsing just means you shouldn't act on the input data before it's fully parsed, not that you can't use some of the input data to decide how to parse other input data. Plenty of useful parsers are context-sensitive.

- **Avoid denormalized representations of data, *especially* if it's mutable.** Duplicating the same data in multiple places introduces a trivially representable illegal state: the places getting out of sync. Strive for a single source of truth.

    - **Keep denormalized representations of data behind abstraction boundaries.** If denormalization is absolutely necessary, use encapsulation to ensure a small, trusted module holds sole responsibility for keeping the representations in sync.

- **Use abstract datatypes to make validators "look like" parsers.** Sometimes, making an illegal state truly unrepresentable is just plain impractical given the tools Haskell provides, such as ensuring an integer is in a particular range. In that case, use an abstract `newtype` with a smart constructor to "fake" a parser from a validator.

As always, use your best judgement. It probably isn't worth breaking out singletons and refactoring your entire application just to get rid of a single `error "impossible"` call somewhere—just make sure to treat those situations like the radioactive substance they are, and handle them with the appropriate care. If all else fails, at least leave a comment to document the invariant for whoever needs to modify the code next.

Recap, reflection, and related reading

That's all, really. Hopefully this blog post proves that taking advantage of the Haskell type system doesn't require a PhD, and it doesn't even require using the latest and greatest of GHC's shiny new language extensions—though they can certainly sometimes help! Sometimes the biggest obstacle to using Haskell to its fullest is simply being aware what options are available, and unfortunately, one downside of Haskell's small community is a relative dearth of resources that document design patterns and techniques that have become tribal knowledge.

None of the ideas in this blog post are new. In fact, the core idea—"write total functions"—is conceptually quite simple. Despite that, I find it remarkably challenging to communicate actionable, practicable details about the way I write Haskell code. It's easy to spend lots of time talking about abstract concepts—many of which are quite valuable!—without communicating anything useful about *process*. My hope is that this is a small step in that direction.

Sadly, I don't know very many other resources on this particular topic, but I do know of one: I never hesitate to recommend Matt Parson's fantastic blog post Type Safety Back and Forth. If you want another accessible perspective on these ideas, including another worked example, I'd highly encourage giving it a read. For a significantly more advanced take on many of these ideas, I can also recommend Matt Noonan's 2018 paper Ghosts of Departed Proofs, which outlines a handful of techniques for capturing more complex invariants in the type system than I have described here.

As a closing note, I want to say that doing the kind of refactoring described in this blog post is not always easy. The examples I've given are simple, but real life is often much less straightforward. Even for those experienced in type-driven design, it can be genuinely difficult to capture certain invariants in the type system, so do not consider it a personal failing if you cannot solve something the way you'd like!

Consider the principles in this blog post ideals to strive for, not strict requirements to meet. All that matters is to try.

1. Technically, in Haskell, this ignores "bottoms," constructions that can inhabit *any* value. These aren't "real" values (unlike `null` in some other languages)—they're things like infinite loops or computations that raise exceptions—and in idiomatic Haskell, we usually try to avoid them, so reasoning that pretends they don't exist still has value. But don't take my word for it—I'll let Danielsson et al. convince you that Fast and Loose Reasoning is Morally Correct. ↩

2. In fact, `Data.List.NonEmpty` already provides a `head` function with this type, but just for the sake of illustration, we'll reimplement it ourselves. ↩

3. Sometimes it is necessary to perform some kind of authorization before parsing user input to avoid denial of service attacks, but that's okay: authorization should have a relatively small surface area, and it shouldn't cause any significant modifications to the state of your system. ↩